DESIGN PATTERNS · BEHAVIORAL | GoF · 13 / 23 | 4-dive study card

Chain

Avoid coupling a request’s sender to its receiver by giving several objects a chance to handle it along a chain.

DIVE 01 · 10 km — Airplane porthole — The big picture & contours
DIVE 02 · 1 km — Train window — Theses, examples, terms
DIVE 03 · 100 m — Park walk — How the pieces relate
DIVE 04 · 1 cm — Microscope — Exact wording & gotchas

Airplane porthole — DIVE 01 · ORIENTATION — ▼ 10 km

A request travels a line of handlers — each either handles it or passes it on, so the sender never needs to know who answers.

— THE CONTOURS

- **Decouples sender & receiver** — the caller just drops the request at the chain’s head
- **Each handler decides** — handle it, pass it on, or both
- **Chain is configurable** — order & membership are set at runtime, not hard-wired

— USE IT WHEN — GoF applicability

- **More than one object may handle** — and which handler responds isn’t known until runtime
- **Issue a request to one of several** — objects without naming the receiver explicitly

Behavioral family: Chain passes a request along · Command reifies it as an object · Mediator centralizes peer talk · Observer broadcasts to many. Chains often carry Commands.

DIVE DEEPER

Train window — DIVE 02 · KEY OBJECTS — ▼ 1 km

— THE THREE PARTS

- **Handler** — defines the handling interface and (often) holds the next link
- **ConcreteHandler** — handles requests it can; otherwise forwards to its successor
- **Client** — builds the chain and sends the request to the first handler

— IN THE WILD

- **Express / Koa middleware** — each layer handles or calls next() — the canonical chain
- **Servlet / ASP.NET filters** — a request passes through a filter pipeline
- **DOM event bubbling** — an event walks up ancestors until handled or stopped
- **Logging appenders** — a record flows to the first handler that accepts its level

```
<> Client → h1.handle(req) → (can’t) → h2.handle(req) → … → a handler responds, or it falls off the end.
```

GoF intent: “Give more than one object a chance to handle the request by passing it along the chain.”

DIVE DEEPER

Park walk — DIVE 03 · CONNECTIONS — ▼ 100 m

Client (sends request) → Handler 1 (handle or pass) → → Handler 2..n (down the chain) → ✓ handled (or unhandled)

— WHY IT FOLLOWS

- → **Successor link ⇒** each handler knows only the next, not the whole pipeline
- → **Sender drops at the head ⇒** it’s decoupled from whoever ends up handling
- → **Dynamic assembly ⇒** add, remove or reorder handlers without touching the sender

— STRUCTURE · UML

handle() either services the request or calls next.handle(req); the client only holds the first Handler.

— WHAT IT BUYS YOU

- ✓ **Reduced coupling —** sender and receivers don’t know each other
- ✓ **Flexible responsibilities —** split and reorder handling among objects at runtime
- ✓ **Object-oriented if/else —** replaces a big conditional with a pipeline of handlers
- ✓ **No guarantee of handling —** a request can fall off the end unanswered
- ✓ **Can be hard to trace —** the runtime path isn’t obvious from the source

Often composed with Composite (a parent is the successor). Handlers frequently carry Commands. Same linked-wrapper shape as Decorator — but a Chain may stop early, a Decorator always delegates.

DIVE DEEPER

Microscope — DIVE 04 · DETAILS — ▼ 1 cm

Definition: “Avoid coupling the sender of a request to its receiver by giving more than one object a chance to handle the request.”
— GoF, “Design Patterns”, 1994

— THE CODE

```
abstract class Handler {
next?: Handler;
setNext(h) { this.next = h; return h; }
handle(req) { return this.next?.handle(req); }
}
class Auth extends Handler {
handle(req) {
if (!req.user) return "401";
return super.handle(req); // pass it on
}
}
```

— VARIANTS

- **Linked handlers** — each holds a successor — the classic form
- **List / pipeline** — an array iterated in order (middleware style)
- **Pass-through chain** — every handler runs; none stops it
- **Short-circuit chain** — the first capable handler stops propagation
- **Tree / bubbling** — an event walks up a parent chain (DOM)
- **Default tail** — a terminal handler guarantees a response

— COMPARE THE ALTERNATIVES

Property	Chain	Decorator	Observer	Mediator
One receiver handles	✓	✗	✗	~
Can stop early	✓	✗	✗	~
Broadcasts to many	✗	✗	✓	✗
Linked-wrapper shape	✓	✓	✗	✗
Decouples sender / receiver	✓	~	✓	✓

Chain = one of many handles · Observer = all are told · Mediator = a hub coordinates · Decorator always delegates.

— NUANCES & GOTCHAS

- **May go unhandled** — add a default tail handler or the request silently drops
- **Order is behavior** — reordering handlers changes outcomes; make it explicit
- **Two stop modes** — short-circuit (first match) vs run-all pipeline; choose deliberately
- **Hard to debug** — the runtime path isn’t visible in the source; log the hops
- **Don’t overuse it** — a simple switch beats a chain for a fixed, small set
- **Shared mutable request** — handlers mutating one context can couple invisibly

— INTERVIEW PITFALLS

Q “Chain vs Decorator?”
Same linked shape; a Decorator always delegates and adds behavior, a Chain handler may stop the request.

Q “Is Express middleware a chain?”
Yes — each handler processes, then calls next(): the textbook Chain of Responsibility.

Q “What if no one handles it?”
It falls off the end; add a default / terminal handler when a response is required.

Q “Short-circuit vs pass-through?”
Short-circuit stops at the first capable handler; pass-through runs every handler as a pipeline.

— WATCH OUT

⚠ **Anti-pattern:** Skip it when exactly one known object handles the request, or the set is small and fixed — a direct call or switch is clearer. Don’t build a chain just to look extensible.

⚡ **Modern take:** HTTP middleware (Express, Koa, NestJS guards & interceptors, ASP.NET) is Chain of Responsibility everywhere — composable pipelines of handlers around a request.